

LAB REPORT: BANKER'S ALGORITHM

1. Introduction

In multiprogramming operating systems, several processes simultaneously compete for limited system resources. If these resources are allocated without proper control, the system may enter a *deadlock*, where a group of processes waits indefinitely for resources held by each other.

The Banker's Algorithm, proposed by Edsger Dijkstra, is a deadlock-avoidance algorithm that checks whether allocating resources to a process leaves the system in a *safe* or *unsafe* state. If the state becomes unsafe, the allocation is denied. This ensures that the system never enters a deadlock.

2. Objectives

- To understand the concept of safe and unsafe states in resource allocation.
- To study and implement the Banker's Algorithm for deadlock avoidance.
- To determine whether a system state is safe by computing safe sequences.
- To verify real-time resource requests using the safety check mechanism.

3. Theory

3.1 Safe and Unsafe States

A system is in a safe state if there exists at least one sequence of processes (safe sequence) such that each process can obtain its maximum required resources, execute, and release resources for the next process. If no such sequence exists, the system is considered unsafe, which may potentially lead to deadlock.

3.2 Data Structures Used in Banker's Algorithm

For n processes and m resource types:

- **Allocation[n][m]**
Amount of each resource currently allocated to each process.
- **Max[n][m]**
Maximum demand of each process.
- **Available[m]**
Number of available instances of each resource.
- **Need[n][m]**
Additional resources each process may still request:

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

3.3 Safety Algorithm

Steps:

1. Initialize $Work = Available$, and $Finish[i] = false$ for all i .
2. Find a process whose $Finish = false$ and $Need \leq Work$.
3. If found:
 - $Work = Work + Allocation$ of that process
 - $Finish = true$
 - Add process to safe sequence
4. If no such process exists and not all $Finish$ are true $\rightarrow$ unsafe state.
5. If all processes finish $\rightarrow$ safe state.

3.4 Resource Request Algorithm

When process P_i requests resources:

1. Check if $Request \leq Need$.
2. Check if $Request \leq Available$.
3. Temporarily allocate the resources:
 - $Available -= Request$
 - $Allocation += Request$
 - $Need -= Request$
4. Run the Safety Algorithm.
5. If system remains safe $\rightarrow$ grant the request; else $\rightarrow$ rollback.

4. Experimental Procedure

1. Input:
 - Number of processes (n) and resources (m)
 - Allocation matrix
 - Max matrix
 - Available vector

2. Compute the Need matrix using the formula.
3. Apply the Safety Algorithm to check whether the current state is safe.
4. Display:
 - Status of the system (Safe / Unsafe)
 - Safe sequence (if safe)

5. Code:

```
#include <bits/stdc++.h>

using namespace std;

void marwan()
{
    int n, m;

    // printf("Enter number of processes and number of resources");

    cin >> n >> m;

    vector<vector<int>> alloc(n, vector<int>(m, 0));
    vector<vector<int>> max(n, vector<int>(m, 0));
    vector<vector<int>> need(n, vector<int>(m, 0));
    vector<int> avail(m, 0);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            cin >> alloc[i][j];
        }
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
```

```

    {
        cin >> max[i][j];
    }
}
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
cerr << "Need Mattrix" << endl;
for (auto &vl : avail)
    cin >> vl;
for (auto row : need)
{
    for (auto vl : row)
        cerr << vl << " ";
    cerr << endl;
}
int count = 0, prevcount = 0;
vector<bool> found(n, false);
vector<int> ans;
for (int k = 0; k < n; k++)
{
    for (int i = 0; i < n; i++)
    {

```

```

bool flag = true;
if (found[i])
    continue;
for (int j = 0; j < m; j++)
{
    if (avail[j] < need[i][j])
    {
        flag = false;
        break;
    }
}
if (flag)
{
    count++;
    found[i] = true;
    ans.push_back(i);
    for (int j = 0; j < m; j++)
        avail[j] += alloc[i][j];
}
}
if (count == n)
{
    cout << "All Processes completed successfully." << endl;
    cout << "Process Order:";
    for (auto vl : ans)
        cout << "p" << vl << " ";
    cout << endl;
}

```

```

        return;
    }
    if (count == prevcount)
    {
        printf("Unsafe State occurred.");
        return;
    }
    prevcount = count;
}
}
int32_t main()
{
    ios_base::sync_with_stdio(false);
    marwan();
    return 0;
}

```

6. Sample Input

```

5 3
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2

```

4 3 3

3 3 2

7. Output:

All Processes completed successfully.

Process Order:p1 p3 p4 p0 p2

8. Conclusion

The Banker's Algorithm provides a reliable method for avoiding deadlock by ensuring the system always remains in a safe state before granting resources. Through the experiment, the structure of resource tables and the concept of a safe sequence were clearly understood. This algorithm is an essential mechanism in operating system resource management.